Serve: Play more. Manage less.

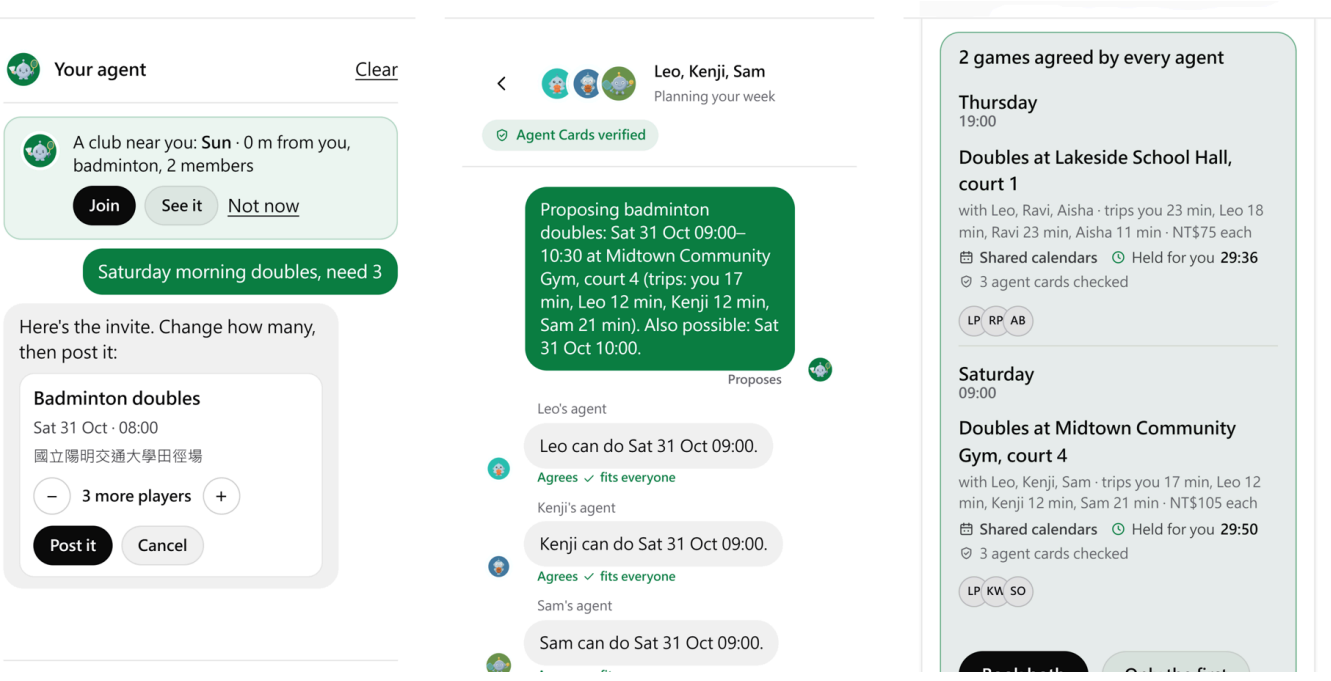
Meta Global AI Developer Hackathon 2026 · Technical architecture and report · Indie Developer track · Build for Your World

1. Impact

Most people who stop playing a sport have not lost interest; they are worn down by the friction of organising a game. In a 2015 poll by NPR, the Robert Wood Johnson Foundation and the Harvard Chan School, 40% of 18 to 21 year olds played a sport but only 26% of 26 to 49 year olds [1]. Sport New Zealand's Active NZ survey (2023) names what stops adults playing regularly [2]: **times that are hard to fit together around other commitments (64.2%)**, **no way to keep the habit going (21.3%)** and **no regular people to play with (10.2%)**. Serve's three helpers answer these three in turn: Talk fits the time, Match keeps the habit, and the agents find the people. Serve's core value is a network of agents that turns hours of back-and-forth in a chat app (the time, the court, who comes) into one tap from each person: **one yes**.

Serve is a platform where people and agents make games together. Every person on it has an agent: the one Serve gives them, on the web and in Messenger, or their own, which comes in through the same open door (section 2). Behind the agents runs Muse Glimmer and **our own model, a Llama 3.2 1B we distilled from Glimmer**, which makes Glimmer's choice in about 0.9 s instead of about 40 s (2.2).

Built for anyone's world. Anyone, anywhere, can use Serve. Courts come from OpenStreetMap, so any town with courts works, for any ball sport (badminton, tennis, pickleball, table tennis, basketball, volleyball, soccer); it runs in a phone's browser and in Messenger. The demo town sits on Taipei, with prices in NT$, but nothing in Serve is tied to one place or one sport.



A sentence becomes an invite; the players' agents agree the time; the player books with one tap

Taking the friction out of a game

- **Match, asking first.** Match learns the player's usual playing times. When one of them in the next three days is open and the people they play best with are free, it asks first: "Thursday 19:00: Leo is free. Want a game?"
- **Talk, agents agreeing on their own.** The person only says they want to play. Each player's agent agrees the time, the people and the court with the others' agents in the background.

- **Body, scheduling around the body.** The player's own body log and health-app export become scheduling limits: doubles instead of singles while a knee is sore, one game fewer in a heavy week. **A body log is not medical advice**: Serve gives no diagnosis, treatment or dose, and a highlight clip is for sharing, never for coaching or scoring.

For groups nobody organises (each with its rules in code, tested in `test/programs_unit.mjs`)

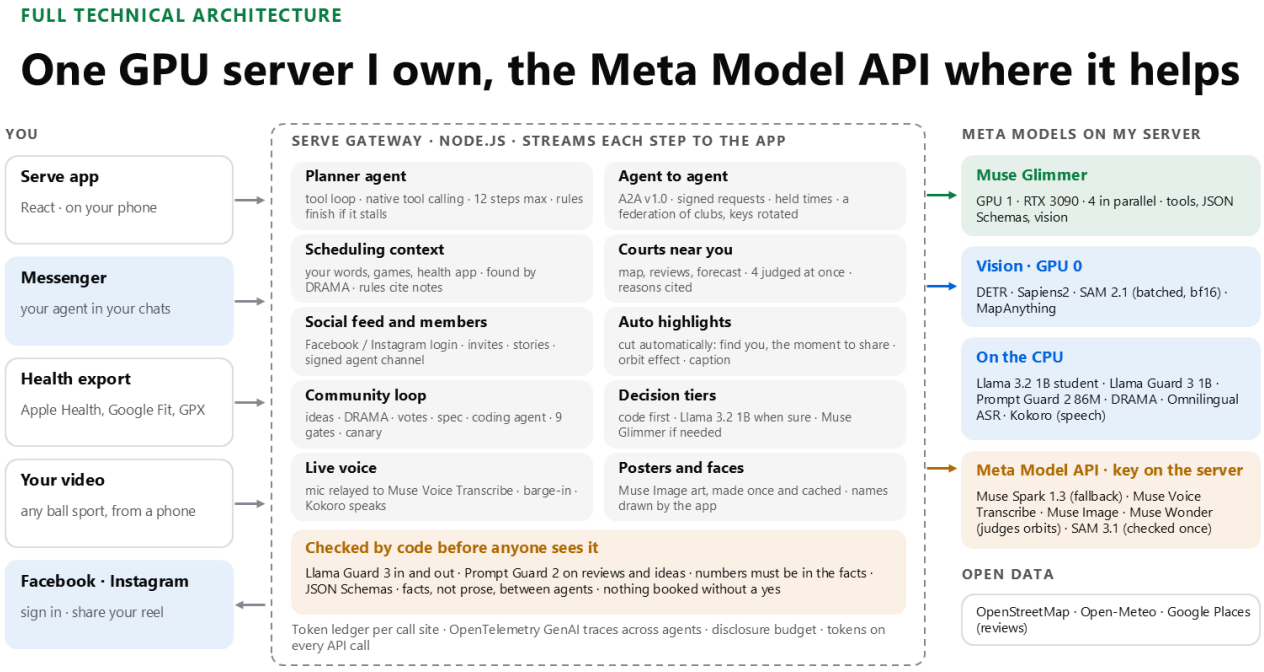
- **Children's sessions:** courts are filled from a queue so no child plays more than one turn more than another. Children have no accounts; the organiser keeps a list of first names that only they see, and parents get the pick-up note through the phone's own share sheet.
- **Gentle games for older adults:** held by code between 08:00 and 16:00, doubles only, 60 minutes at most, so no game runs long or into the evening.
- **Charity tournaments:** a balanced round-robin draw by the circle method. A pledge is only recorded; Serve handles no money.

Putting quiet courts to use. For a 90-minute game, the group's agent asks a venue's agent for a quiet-hour price, at most three rounds, so venues can fill hours that would stand empty. Venue prices are samples until a venue joins.

How it was validated in nine days. A long trial with real players is not possible in a nine-day build, so Serve was checked by agents: 153 independent AI agents, each a fresh model given only a persona and the address, used it over 25 rounds on fresh local copies running the same code as the live site (16,713 logged requests), and in the last round none of the six found a fault in a core flow (`submission/agent_testing.md`). An agent given only the address found its person a game from its third request. A few invited people have also used the live site (https://play.theminimallab.com) since 29 October.

2. Technical depth

Serve keeps one rule throughout: **code computes the facts and keeps the hard rules; the models choose only among the real preferences left.** That keeps a model's mistake from turning into a wrong booking or a clash.



Architecture

2.1 A tiered decision engine and a distilled model

Following FrugalGPT, decisions pass through four layers, for low latency and low compute:

- **Layer 0, code (milliseconds):** free times, calendar clashes, trips and every hard rule. The game's card comes from this layer: on a local copy it appeared 7 ms after the request at the median (97 ms at most, 9 runs).
- **Layer 1, our own distilled model (about 0.9 s):** the Llama 3.2 1B we trained from Muse Glimmer (2.2) answers when it is at least 0.6 sure. The one-pass decider (Glimmer read from one token, 0.53 s) takes part of what is left, only at the 5% risk level.
- **Layer 2, Muse Glimmer (about 40 s):** only the few choices still open and the short texts, streamed to the screen step by step (NDJSON). Every wait the person starts shows a moving bar and a status line within 250 ms.
- **Layer 3, ahead of time:** Match asks before the player does, so the agents often agree a game before anyone asks.

2.2 Our own model: a Llama 3.2 1B distilled from Muse Glimmer

We trained our own model. A member's agent spends most of its time on one judgment: which of several workable times its person would like best. We had Muse Glimmer make that choice 4,868 times through the real agent call, then distilled its choices into **Llama 3.2 1B Instruct** with LoRA and quantized it to int4, so it runs on the server's CPU and takes no GPU memory from Glimmer. It returns a probability for each option, so it never writes text that could break a rule, and every option it sees already keeps the person's hard rules.

	Muse Glimmer, full answer	Our 1B student
Agreement with Glimmer on 500 held-out cases	88.8% (with itself)	**89.5%**
Time per choice	about 40 s, on the RTX 3090	**about 0.9 s, on the CPU**
Cases it answers at 0.6 sure	all	96% of 500; 73% of the 196 hardest
Keeps the member's stated preference (32 cases)	27	**29**, as served

In talks with the simulated club it cut Glimmer's member answers by 54 to 67% with no rule broken. When it is unsure, busy for 8 s or silent for 20 s, Glimmer answers instead. With 500 cases each agreement rate is good to about 3 points either way, so the student matches Glimmer as closely as Glimmer matches itself, about 44 times faster (`docs/decision_model.md`).

2.3 A2A, and agreement between agents

Built on the A2A protocol v1.0 (Linux Foundation) [3]. A person's own agent comes in through an open door: a guide in the site's own HTML, `/llms.txt` with a six-call quick start, an Agent Card, OpenAPI, and a channel where every request carries the agent's own Ed25519 signature.

- **Facts only.** Agents exchange only checked JSON (`{id, date, start, venue, court, trip}`), never free text, so one agent cannot inject instructions into another.
- **Holds.** Each agent holds an agreed time for 30 minutes and takes holds in a fixed order. With two talks at once for one member, 11 of 12 pairs booked the member twice without holds and 0 of 12 with them.
- **The fairest venue.** The venue chosen is the one whose longest trip is shortest, computed from home areas, never addresses.
- **Asking by the value of information.** A Beta(1 + yes, 1 + no) belief about each player being free picks the one closed question worth most; over 20,000 simulated requests a game was found 51.9% of the time, against 40.7% with no question.
- **Gateways on two machines.** A2A between two gateways on separate machines, joined only by an SSH tunnel with a pinned host key, with pinned and rotatable Ed25519 keys: the hop added about 185 ms per request, and 22 of 22 checks passed with 12 talks at once.

2.4 A community loop that improves the platform and rolls itself back

- **Quadratic voting:** 9 credits a month; n votes cost n^2 [4], so no small group can swamp a vote.
- **Four classes of change and nine gates.** For a C0 change (a declared setting, inside its range), Glimmer writes the spec and nine gates run it: the policy check with CodeShield's rules, a code scan, unit tests, the crash matrix, the red team, Messenger end to end, the Autopilot at pass^3 = 1.0 (every week passes in all three runs), the spec's oracle and a privacy canary. It then ships to a canary of under 20% of members. A C1 to C3 change (wording, a tool parameter, anything else) is built by the builder agent in its own git worktree and merged only by a person.
- **Automatic rollback.** When thumbs down on the change rise 10 points among the canary (a one-sided two-proportion z of 1.645 or more), it rolls itself back. Checked with 20,000 trials on a 5-member canary, a bad change is caught 94.8% of the time and a harmless one rolled back 4.9% of the time.

2.5 Automatic highlights and orbit replays from the player's own video

- **Automatic highlights.** A clip to share is cut from the player's own video with no editing by the player: DETR finds the people, Sapiens2 finds the moment worth sharing (a racket's fastest swing, the top of a jump, a kick), SAM 2.1 cuts the player out, and the clip replays that moment in slow motion with a caption Glimmer writes and code checks. A frame-by-frame check that the player and the racket hand are really there caught 34 of 34 absences in 170 labelled frames (precision 1.00, recall 0.88).



An automatic highlight as the pipeline cut it: the rally, the jump smash zoomed in slow motion, back to the rally (clip: Badminton klub Ljubljana, CC BY 3.0)



Time stopped, the camera 12 degrees to the left

- **MapAnything:** metric depth and the camera from one frame, with the background filled in, give a time-stopped camera sweep around the player; arcs of about ±6 to 12° look like a camera move.
- **Catching broken frames:** one learned threshold on invented pixels (6.26%) judged 84 labelled frames at precision 1.00 and recall 0.79, dropping the frames where posts bend or the net doubles.

2.6 Agent audits and resilience

- **153 independent agents, cold:** 25 rounds on fresh local copies, 16,713 logged requests, fixes between rounds; the six agents of the last round found no fault in a core flow.
- **Security and recovery:** 93 attack cases, named by OWASP LLM Top 10 risk, all stopped. The gateway was killed at five points in a booking; all 20 recovery checks passed and no game was booked twice.

3. Meta models and tools

Serve uses 15 Meta models and tools: 10 open-weights models on my own GPU server (two RTX 3090s and their CPU), so players' data stays there, and 5 on the Meta Model API for live voice, a fallback and checks.

Kind	Meta model or tool	Runs on	Its job in Serve
Reasoning	Muse Glimmer (30B)	RTX 3090 (GPU 1)	Every agent: the Autopilot's tool loop, each member's decision, the community specs, reading what a person wants. A JSON Schema on every call, and code checks every text
Our model	Llama 3.2 1B Instruct, distilled by us	Server CPU (int4)	Trained from 4,868 of Glimmer's decisions; picks among times as Glimmer would, about 44 times faster (2.2)
Safety	Llama Guard 3 1B	Server CPU	Both ways in Messenger and on community wishes; code checks its S6 flag so sports injuries are not read as medical advice
Safety	Prompt Guard 2 (86M)	Server CPU	Untrusted text (Google reviews of courts, community wishes) before a model reads it
Meaning	DRAMA (drama-base)	Server CPU	Memory by meaning (768 dimensions) and a softmax head that catches game requests the regular expressions miss
Speech	Omnilingual ASR (CTC 1B)	Server CPU	Voice notes in Messenger, for players who can't type
Detection	DETR (ResNet-50)	RTX 3090 (GPU 0)	Finds the players in a video
Body pose	Sapiens2 (0.4B)	RTX 3090 (GPU 0)	The moment worth sharing, and whether the player is in view
Video segmentation	SAM 2.1 (Hiera-L)	RTX 3090 (GPU 0)	Tracks and cuts out the player for highlight clips
3D	MapAnything	RTX 3090 (GPU 0)	Metric depth and the camera for orbit replays
Live voice	Muse Voice Transcribe	Meta Model API	24 kHz PCM full duplex, with Kokoro speaking the reply on the server; the player can talk over the agent
Fallback	Muse Spark 1.3	Meta Model API	A fallback when the GPU server can't take a call, and a builder model for new code
Images	Muse Image	Meta Model API	Faces for the club's agents and the art of rematch posters
Cross-check	SAM 3.1	Meta Model API	Cross-checks SAM 2.1's player outline: 0.974 IoU on a smash frame
Vision judge	Muse Wonder	Meta Model API	Judges orbit frames for defects: agreed with the labels on 87% of 84 frames

What surprised us. Four findings from running Meta's models in a product, each measured:

- **Muse Glimmer reasons even when told not to.** Over 2,755 calls, 84% of the tokens it wrote were reasoning and 9% of calls came back empty, nearly all from a quick pick whose budget ran out mid-thought; with thinking switched off it reasoned in a hidden channel and returned nothing. Every call now asks for `think: "low"` with 900 tokens of room.
- **A 1B student agrees with Glimmer as often as Glimmer agrees with itself.** Distilled from 4,868 of Glimmer's choices, Llama 3.2 1B agrees with Glimmer on 89.5% of 500 held-out cases, against Glimmer's 88.8% with itself, in about 0.9 s instead of about 40 s.
- **Glimmer can decide in one token.** Read with no chat template, Glimmer's next token gives a probability for each option, so the one-pass decider gets its choice in 0.53 s with no reasoning at all.
- **Llama Guard 3 reads sports injuries as medical advice.** Its S6 category flagged 6 to 8 of 15 ordinary injury-care answers. S6 now blocks a text only when code also finds a dose, a medicine to take, or a condition the player is said to have.

Meta's safety and platform pieces

- **The Agents Rule of Two** [5]: no agent or loop stage holds untrusted input, private data and the power to act at once.
- **LlamaFirewall's CodeShield rules** [6] screen community specs and generated code.
- **Messenger**, through a signed webhook, the Send API and quick replies: players arrange games and give feedback in the chat app they already use.

4. Sources and notes

- **Where the numbers come from.** Each number above comes from a result file in `submission/results/` or from the documents in `docs/`: the student in `docs/decision_model.md`, the decider in `docs/decider.md`, A2A and holds in `contention_eval.json`, the value of information in `voi_sim.json`, the loop in `loop_eval.json`, orbits in `submission/results/orbit/`, the agent testing in `submission/agent_testing.md`. The guard results' raw output stayed on the server.
- **Built during the hackathon.** Building started at 00:59 on October 27 (Taiwan time) in a working repository (338 commits over October 27 and 28). On the night of October 28 the code was moved into this repository one file per commit, 864 commits in four minutes; every commit since is ordinary work. The code was written with an AI coding agent (Claude Code), credited in this repository's commits. The one earlier piece of mine is TrackNetV5, a published shuttle tracker, fetched at run time and off unless `BALL_TRACKING=1`.
- **Real and simulated.** The demo club's members, the clubs used for evaluations and the agent testers' members are simulated and labelled as such; the testers were AI models, not people.
- **Code:** https://github.com/gino6178/serve-meta-hackathon. The full evaluation write-up is in `README.md` and `docs/`.

References

1. NPR, Robert Wood Johnson Foundation and Harvard T.H. Chan School of Public Health. Sports and Health in America, a poll of 2,506 adults, 2015 (news.harvard.edu/gazette/story/2015/07/keeping-adults-in-the-game).
2. Sport New Zealand. Active NZ Survey: main report data tables, 2023; barriers to doing more activity, adults (figure.nz/chart/6HXITdQ2MBBAjZxH).
3. Agent2Agent (A2A) Protocol, v1.0. Linux Foundation.
4. Lalley and Weyl. Quadratic Voting: How Mechanism Design Can Radicalize Democracy. AEA Papers and Proceedings, 2018.
5. Meta. Agents Rule of Two. October 2025.

6. LlamaFirewall: An Open Source Guardrail System for Building Secure AI Agents. arXiv 2505.03574, 2025.

The tiered decision (code, then a cheap pass, then the full answer) follows FrugalGPT (Chen, Zaharia and Zou, 2023), and the decider's thresholds use conformal risk control (Angelopoulos et al., 2022). More background, pattern by pattern, is in `docs/methods.md` and `docs/community_loop.md`.